

Laboratorium 1: Wprowadzenie do przedmiotu

Opracowanie zestawu zadań (wersja programistyczna)

Kacper Bednarczuk

Część 1: Implementacja słowników kodowania

Słownik Morse'a

Słownik `morseDict` zawiera mapowanie dla wszystkich 26 liter alfabetu łacińskiego (A-Z) oraz cyfr (0-9). Każda litera jest reprezentowana jako ciąg kropek (`.`) i kresek (`-`).

Słownik Braille'a

Słownik `brailleDict` reprezentuje każdy znak jako 6-punktową celę Braille'a. Każda cela to tablica 6 wartości logicznych (`true / false`), gdzie `true` oznacza wypukły punkt.

Układ punktów w celi:

```
1 • • 4
2 • • 5
3 • • 6
```

Przykład dla litery 'A': `[true, false, false, false, false, false]` - tylko punkt 1 jest wypukły.

Uwaga

W pełnym systemie Braille'a istnieje specjalny znak kapitalizacji, który poprzedza wielkie litery. W tej uproszczonej implementacji wszystkie litery są traktowane jednakowo (jak małe litery), więc znak kapitalizacji nie jest używany. Sam tekst jest natomiast automatycznie konwertowany na wielkie litery ze względów technicznych (słowniki zawierają tylko znaki A-Z).

Znak liczby w Braille'u

Przed sekwencją cyfr w Braille'u musi pojawić się specjalny znak liczby `[false, false, true, true, true, true]` (punkty 3, 4, 5, 6). Dzięki temu cyfry mogą używać tych samych wzorów co litery A-J.

Część 2: Funkcje kodowania i dekodowania

Kod Morse'a

`encodeToMorse(plainText)`

- Konwertuje tekst na wielkie litery
- Koduje każdy znak do odpowiedniego kodu Morse'a
- Oddziela litery jedną spacją (' ')
- Oddziela słowa ukośnikiem pomiędzy dwoma spacjami (' / ')

Wyjaśnienie

Jako separator słów wykorzystałem ' / ' zamiast sugerowanych w treści trzech spacji, aby zachować spójność z tłumaczem online poleconym na platformie Wirtualny Kampus.

`decodeFromMorse(morseCode)`

- Tworzy odwrócony słownik (kod -> litera)
- Dzieli tekst na słowa (separator: dwie spacje i ukośnik w środku)
- Dzieli słowa na znaki (separator: jedna spacja)
- Dekoduje każdy znak i składa z powrotem tekst

Braille

`encodeToBraille(plainText)`

- Konwertuje tekst na wielkie litery
- Śledzi "tryb liczby" (`inNumberMode`)
- Przed pierwszą cyfrą w sekwencji dodaje znak liczby
- Spacja kończy tryb liczby
- Zwraca tablicę cel Braille'a

`decodeFromBraille(brailleCode)`

- Używa funkcji pomocniczej `cellsEqual()` do porównywania cel
 - Rozpoznaje znak liczby i przełącza tryb dekodowania
 - Pusta cela `[false, false, false, false, false, false]` = spacja
 - W trybie liczby dekoduje cyfry, poza nim - litery
-

Część 3: Analiza weryfikacyjna

Funkcja `analyseBraille(brailleCode)` oblicza statystyki dla zakodowanego tekstu w Braille'u:

Waga Hamminga

Liczba wypukłych punktów (wartości `true`) w każdej celi. Im więcej punktów, tym trudniejszy odczyt za pomocą dotyku.

Średnia waga Hamminga

Suma wszystkich punktów podzielona przez liczbę cel. Pokazuje przeciętną "gęstość" punktów w tekście.

Procent cel o wysokiej złożoności

Procent cel używających 5 lub 6 punktów (najtrudniejsze do rozróżnienia dotykiem).

Testy i weryfikacja

Wyjaśnienie

Do testów wykorzystałem ciąg znaków "QWERTY 123" przede wszystkim ze względu na to, że moje imię i nazwisko nie zawiera liter o wysokiej wadze Hamminga (Q, Y), a zależało mi na zaprezentowaniu analizy w całości.

Kod Morse'a

Wyjście z konsoli Node.js:

```
===== TEST MORSE =====  
Tekst: QWERTY 123  
Morse: --.- .-- . -. - -. / .---- ..--- ...--  
Dekodowany: QWERTY 123
```

Wynik z tłumacza online (<https://alfabetmorsa.pl/tlumacz-translator-alfabetu-morsea-online>):

Tekst w alfabecie polskim do przetłumaczenia na alfabet Morse'a:

QWERTY 123

↓ tłumacz na alfabet Morse'a ↓

Tekst w alfabecie Morse'a do przetłumaczenia na alfabet polski (rozdzielony spacją):

...- .- .- .- .- .- / .- .- .- .- .- .-

↑ tłumacz na zwykły alfabet ↑

Braille

Wyjście z konsoli Node.js:

```
===== TEST BRAILLE =====
```

```
Tekst: QWERTY 123
```

```
Braille:
```

```
Cela 1: [ true, true, true, true, true, false ]
Cela 2: [ false, true, false, true, true, true ]
Cela 3: [ true, false, false, false, true, false ]
Cela 4: [ true, true, true, false, true, false ]
Cela 5: [ false, true, true, true, true, false ]
Cela 6: [ true, false, true, true, true, true ]
Cela 7: [ false, false, false, false, false, false ]
Cela 8: [ false, false, true, true, true, true ]
Cela 9: [ true, false, false, false, false, false ]
Cela 10: [ true, true, false, false, false, false ]
Cela 11: [ true, false, false, true, false, false ]
```

```
Dekodowany: QWERTY 123
```

Wynik z tłumacza online (<https://www.braille.edu.pl/mobile/index.html>):



Analiza weryfikacyjna

Wyjście z konsoli Node.js:

```
===== ANALIZA BRAILLE =====  
Wszystkie cele:  
1. true,true,true,true,true,false -> waga: 5  
2. false,true,false,true,true,true -> waga: 4  
3. true,false,false,false,true,false -> waga: 2  
4. true,true,true,false,true,false -> waga: 4  
5. false,true,true,true,true,false -> waga: 4  
6. true,false,true,true,true,true -> waga: 5  
7. false,false,false,false,false,false -> waga: 0  
8. false,false,true,true,true,true -> waga: 4  
9. true,false,false,false,false,false -> waga: 1  
10. true,true,false,false,false,false -> waga: 2  
11. true,false,false,true,false,false -> waga: 2  
Średnia waga Hamminga: 3.00  
Procent cel o wysokiej złożoności (5-6 punktów): 18.18%
```

Uwaga

W analizie liczony jest również znak liczby oraz spacja. Dlatego dla podanego tekstu wynik wynosi ~18.18% ($\frac{2}{11}$), a nie, tak jak mogłoby się wydawać ~22.22% ($\frac{2}{9}$).

Wnioski

1. **Morse** jest prostym kodem o zmiennej długości - krótkie znaki dla częstych liter (E = .)
2. **Braille** używa stałej długości (6 punktów), ale różnej liczby wypukłości
3. **Analiza Braille'a** pokazuje, że większość znaków używa 2-3 punktów, co ułatwia czytanie

Możliwe rozszerzenia

- Obsługa znaków interpunkcyjnych
- Pełna obsługa wielkich/małych liter w Braille'u
- Wizualizacja graficzna cel Braille'a (np. ASCII art)